

DETAIL MINI PROJECT

Area: Blockchain

Mata Kuliah: Komputasi Berbasis Jaringan — Program S2 Tesis

Timeline: 3 Pekan Implementasi + 1 Pekan Pengujian Komprehensif

Dokumen ini memuat detail teknis untuk 4 topik mini project, masing-masing mencakup deskripsi, infrastruktur, langkah pengerjaan, dan skenario uji coba komprehensif beserta template pengumpulan data.

TOPIK 3A

Desain dan Evaluasi Reputation-Based Consensus untuk Permissioned Blockchain pada Skenario Supply Chain

1. Deskripsi

Mekanisme konsensus di mana hak validasi ditentukan oleh skor reputasi node. Node yang sering mengajukan transaksi invalid kehilangan hak voting secara gradual. Custom lightweight blockchain Python. Perbandingan 4 mekanisme: PBFT (baseline), Proof-of-Authority (PoA), simplified DPoS, dan Reputation-Based (proposal). Bisa jalan di Google Colab tanpa Docker.

2. Infrastruktur

- 7 Python processes (5 honest + 2 byzantine) via socket localhost.
- Blockchain Core (~300 LOC), Consensus Module (~200 LOC, 4 implementasi), Reputation Manager (~100 LOC).
- Byzantine Simulator: 3 behavior (invalid TX, inconsistent vote, delay). Transaction Generator: sebagian sengaja invalid.

3. Langkah

Pekan 1: Blockchain core + PBFT + process setup. Pekan 2: Reputation + PoA + DPoS + Byzantine. Pekan 3: Integrasi + tuning. Pekan 4: Eksekusi + report.

4. Skenario Uji Coba dan Matriks Evaluasi

4.1 Variabel Tetap

- 7 nodes, block size 50 tx, block interval 5 detik, EMA $\alpha=0.9$, threshold=0.3, seed tetap, warm-up 10 block pertama di-skip, genesis reset antar run, TX rate: 50 tx/s.

4.2 Variabel Manipulasi

Variabel	Level
Konsensus	PBFT, PoA, DPoS, Reputation-Based
Jumlah Byzantine Nodes	0 (all honest), 1 node, 2 nodes
Byzantine Behavior	Invalid-TX-Only, Inconsistent-Vote, Mixed (keduanya)
Invalid TX Rate	0%, 10%, 30%

Total: 4 konsensus $\times$ 3 byzantine count $\times$ 3 behavior $\times$ 3 invalid rate = 108 kombinasi. Namun 0 byzantine tidak terpengaruh behavior dan invalid rate, sehingga efektif = $(4 \times 1) + (4 \times 2 \times 3 \times 3) = 4 + 72 = 76 \times 5 \text{ rep} = 380 \text{ run}$. Prioritas: 2 byzantine + Mixed behavior = $4 \times 3 \text{ invalid rate} = 12 \times 5 = 60 \text{ run} (\sim 10 \text{ jam})$.

4.3 Variabel Respon

Variabel	Satuan	Pengukuran
Throughput (TPS)	tx/s	Transaksi valid committed per detik

Consensus Latency	ms	Dari proposal hingga finality
Block Finality Rate	%	Block finalized / total proposed
Byzantine Detection Rate	%	Byzantine node yang reputasinya < threshold
False Positive Rate	%	Honest node yang turun salah
Reputation Convergence	blocks	Block hingga byzantine teridentifikasi
Chain Consistency	boolean	Semua honest node chain identik
Message Complexity	msg/block	Total pesan konsensus per block

4.4 Template Pengumpulan Data

Isi setiap sel dengan rata-rata $\pm$ standar deviasi dari 5 repetisi.

Tabel 4.4a — Byzantine: 0 nodes (All Honest), Invalid TX Rate: 0%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Consensus Latency (ms)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Finality Rate (%)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Chain Consistent	___	___	___	___
Msg/Block	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___

Tabel 4.4b — Byzantine: 1 node, Behavior: Invalid-TX-Only, Invalid Rate: 10%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Consensus Latency (ms)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Finality Rate (%)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Detection Rate (%)	—	—	—	___ $\pm$ ___
False Positive (%)	—	—	—	___ $\pm$ ___
Convergence (blocks)	—	—	—	___ $\pm$ ___
Chain Consistent	___	___	___	___
Msg/Block	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___

Tabel 4.4c — Byzantine: 1 node, Behavior: Invalid-TX-Only, Invalid Rate: 30%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Consensus Latency (ms)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Finality Rate (%)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___
Detection Rate (%)	—	—	—	___ $\pm$ ___
Convergence (blocks)	—	—	—	___ $\pm$ ___
Chain Consistent	___	___	___	___

Tabel 4.4d — Byzantine: 2 nodes, Behavior: Invalid-TX-Only, Invalid Rate: 10%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___	___ $\pm$ ___

Consensus Latency (ms)	___±___	___±___	___±___	___±___
Finality Rate (%)	___±___	___±___	___±___	___±___
Detection Rate (%)	—	—	—	___±___
False Positive (%)	—	—	—	___±___
Convergence (blocks)	—	—	—	___±___
Chain Consistent	—	—	—	—
Msg/Block	___±___	___±___	___±___	___±___

Tabel 4.4e — Byzantine: 2 nodes, Behavior: Invalid-TX-Only, Invalid Rate: 30%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___±___	___±___	___±___	___±___
Finality Rate (%)	___±___	___±___	___±___	___±___
Detection Rate (%)	—	—	—	___±___
Convergence (blocks)	—	—	—	___±___
Chain Consistent	—	—	—	—

Tabel 4.4f — Byzantine: 2 nodes, Behavior: Inconsistent-Vote, Invalid Rate: 0%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___±___	___±___	___±___	___±___
Consensus Latency (ms)	___±___	___±___	___±___	___±___
Finality Rate (%)	___±___	___±___	___±___	___±___
Detection Rate (%)	—	—	—	___±___
Convergence (blocks)	—	—	—	___±___
Chain Consistent	—	—	—	—

Tabel 4.4g — Byzantine: 2 nodes, Behavior: Mixed (Invalid-TX + Inconsistent-Vote), Invalid Rate: 10%

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___±___	___±___	___±___	___±___
Consensus Latency (ms)	___±___	___±___	___±___	___±___
Finality Rate (%)	___±___	___±___	___±___	___±___
Detection Rate (%)	—	—	—	___±___
False Positive (%)	—	—	—	___±___
Convergence (blocks)	—	—	—	___±___
Chain Consistent	—	—	—	—
Msg/Block	___±___	___±___	___±___	___±___

Tabel 4.4h — Byzantine: 2 nodes, Behavior: Mixed, Invalid Rate: 30% (WORST CASE)

Metrik	PBFT	PoA	DPoS	Reputation
TPS (tx/s)	___±___	___±___	___±___	___±___
Consensus Latency (ms)	___±___	___±___	___±___	___±___
Finality Rate (%)	___±___	___±___	___±___	___±___

Detection Rate (%)	—	—	—	___±___
False Positive (%)	—	—	—	___±___
Convergence (blocks)	—	—	—	___±___
Chain Consistent	___	___	___	___
Msg/Block	___±___	___±___	___±___	___±___

4.5 Uji Statistik

- Mann-Whitney U per pasangan konsensus. Kruskal-Wallis: 4 sekaligus.
- Survival analysis (Kaplan-Meier): time-to-detection byzantine per mechanism.
- Chi-square: chain consistency rate antar konsensus.
- Sensitivity: EMA $\alpha = \{0.7, 0.8, 0.9, 0.95\}$ pada Reputation + 2 byz + Mixed 30%.

4.6 Visualisasi

- Time-series reputation score per node (1 plot per konsensus pada worst case 4.4h).
- Bar chart TPS + finality per konsensus, grouped per byzantine scenario.
- Line chart validator set size over blocks.
- Kaplan-Meier curve: time-to-detection per mechanism.

TOPIK 3B

Gas Optimization Framework untuk Smart Contract Solidity Menggunakan Static Analysis

1. Deskripsi

Tool analisis statik yang mendeteksi 6 anti-pattern boros gas pada Solidity, memberikan rekomendasi refactoring, dan mengukur penghematan gas empiris. Diuji pada 50–100 contracts dari Etherscan. Head-to-head vs Slither.

2. Infrastruktur

- Python AST parser + solc + Hardhat in-memory testnet. Dataset: 50–100 verified contracts, stratified 10 per domain.
- Gas 100% deterministik di Hardhat. Bisa di Colab.

3. Langkah

Pekan 1: solc + Hardhat + download contracts + AST parser. Pekan 2: 6 pattern detectors + gas estimator. Pekan 3: Refactoring recommender + Slither comparison. Pekan 4: Full analysis + report.

4. Skenario Uji Coba

4.1 Variabel Tetap

- solc 0.8.20, Hardhat network, optimization off, gas measurement 3x median, manual audit 20 contracts (2 reviewer, Cohen's kappa).

4.2 Variabel Manipulasi

Variabel	Level
Anti-Pattern Type	Redundant SLOAD, Unoptimized Loop, String vs Bytes32, Public vs External, Unchecked Arithmetic, Dead Code
Contract Domain	DeFi, NFT, Token (ERC-20/721), Governance, Utility
Contract Complexity	Simple (<100 LOC), Medium (100–500 LOC), Complex (500+ LOC)

4.3 Variabel Respon

Variabel	Satuan	Pengukuran
Precision (per pattern)	%	Correctly detected / total detected
Recall (per pattern)	%	Detected / actual (manual audit)
Gas Saved per Pattern	gas	Original vs refactored
Gas Saved Percentage	%	$(\text{Saved} / \text{original}) \times 100$
Auto-Refactor Success	%	Compiles + tests pass
Analysis Time	detik	Wall-clock per contract
False Positive Rate	%	Incorrect / total detected

4.4 Template Pengumpulan Data

Tabel 4.4a — Detection Accuracy per Anti-Pattern (dari 20 manually-audited contracts)

Anti-Pattern	True Pos	False Pos	False Neg	Precision (%)	Recall (%)	Contracts Affected
Redundant SLOAD	___	___	___	___	___	___/20
Unoptimized Loop	___	___	___	___	___	___/20
String vs Bytes32	___	___	___	___	___	___/20
Public vs External	___	___	___	___	___	___/20
Unchecked Arithmetic	___	___	___	___	___	___/20
Dead Code	___	___	___	___	___	___/20

Tabel 4.4b — Gas Savings per Anti-Pattern (dari 50 contracts, Hardhat measurement)

Anti-Pattern	Contracts w/ Pattern	Avg Gas Before	Avg Gas After	Avg Saved (%)	Max Saved (%)	Refactor Success (%)
Redundant SLOAD	___/50	___	___	___±___	___	___
Unoptimized Loop	___/50	___	___	___±___	___	___
String vs Bytes32	___/50	___	___	___±___	___	___
Public vs External	___/50	___	___	___±___	___	___
Unchecked Arithmetic	___/50	___	___	___±___	___	___
Dead Code	___/50	___	___	___±___	___	___

Tabel 4.4c — Cross-Domain Analysis: Pattern Distribution per Domain (50 contracts)

Anti-Pattern	DeFi (10)	NFT (10)	Token (10)	Governance (10)	Utility (10)
Redundant SLOAD	___	___	___	___	___
Unoptimized Loop	___	___	___	___	___
String vs Bytes32	___	___	___	___	___
Public vs External	___	___	___	___	___
Unchecked Arithmetic	___	___	___	___	___
Dead Code	___	___	___	___	___
Total Patterns	___	___	___	___	___
Avg Gas Saved (%)	___	___	___	___	___

Tabel 4.4d — Complexity Scalability: Akurasi per Complexity Level

Metrik	Simple (<100 LOC)	Medium (100–500)	Complex (500+)
Avg Patterns Detected	___±___	___±___	___±___
Overall Precision (%)	___	___	___
Overall Recall (%)	___	___	___
Analysis Time (s)	___±___	___±___	___±___
False Positive Rate (%)	___	___	___

Tabel 4.4e — Head-to-Head vs Slither (50 contracts)

Metrik	Our Tool	Slither	Overlap
Total Patterns Detected	—	—	—
Unique to Our Tool	—	—	—
Unique to Slither	—	—	—
Precision (on shared patterns)	—	—	—
Gas Quantification	Ya (per pattern)	Tidak	—
Avg Analysis Time (s)	—	—	—

4.5 Uji Statistik

- Cohen's kappa: inter-rater agreement manual audit.
- McNemar's test: our tool vs Slither per contract (paired binary).
- Wilcoxon signed-rank: gas before vs after per contract.
- Kruskal-Wallis: gas savings across 5 domains.
- Spearman correlation: LOC vs total patterns detected.

4.6 Visualisasi

- Bar chart gas savings per pattern type. Venn diagram detection overlap vs Slither.
- Heatmap: pattern frequency per domain. Scatter: LOC vs patterns detected.

TOPIK 3C

Analisis Performa State Channel untuk Transaksi Mikro pada Ethereum Layer-2

1. Deskripsi

State channel on-chain/off-chain untuk micro-TX. Hardhat testnet. Fokus: break-even point, dispute cost, settlement frequency impact. Perbandingan: All On-Chain, State Channel, Multi-Hop Channel.

2. Infrastruktur

- Solidity (~150 LOC) + off-chain client (Node.js/Python ~200 LOC) + Hardhat in-memory. Gas 100% deterministik — 3 repetisi cukup. Total: ~48 menit.

3. Langkah

Pekan 1: Smart contract + Hardhat + tests. Pekan 2: Off-chain client + settlement + dispute. Pekan 3: Gas metering + break-even + multi-hop. Pekan 4: Experiment + report.

4. Skenario Uji Coba

4.1 Variabel Tetap

- Hardhat, solc 0.8.20, 2 participants, TX amount uniform 0.001–0.1 ETH, fixed mnemonic.

4.2 Variabel Manipulasi

Variabel	Level
Mode Transaksi	All On-Chain (baseline), State Channel, Multi-Hop Channel
Jumlah TX per Channel	10, 50, 100, 500, 1000
Settlement Frequency	Every 10 tx, Every 50 tx, At close only
Dispute Scenario	No dispute, 1 dispute mid-channel, Dispute at close

Total: 3 mode × 5 TX count × 3 settlement × 3 dispute = 135 kombinasi. Namun On-Chain tidak terpengaruh settlement/dispute. Efektif: 5 + (2×5×3×3) = 5 + 90 = 95 × 3 rep = 285 run. Karena deterministik dan per run ~10 detik, total ~48 menit.

4.3 Variabel Respon

Variabel	Satuan	Pengukuran
Total Gas Cost	gas	Aggregate gas seluruh lifecycle
Gas per Micro-TX	gas/tx	Total gas / jumlah TX
Off-Chain Throughput	tx/s	TX per detik off-chain
Break-Even Point	tx count	Min TX di mana channel lebih murah
Dispute Gas Cost	gas	Extra gas untuk dispute
Settlement Overhead	gas	Gas untuk periodic settlement

Channel Open/Close Gas	gas	Fixed cost
------------------------	-----	------------

4.4 Template Pengumpulan Data

Tabel 4.4a — Settlement: At Close Only, Dispute: No Dispute

TX Count	Gas On-Chain	Gas Channel	Gas Multi-Hop	Gas/TX On-Chain	Gas/TX Channel	Break-Even?
10	—	—	—	—	—	Ya/Tidak
50	—	—	—	—	—	Ya/Tidak
100	—	—	—	—	—	Ya/Tidak
500	—	—	—	—	—	Ya/Tidak
1000	—	—	—	—	—	Ya/Tidak

Tabel 4.4b — Settlement: Every 10 TX, Dispute: No Dispute

TX Count	Gas On-Chain	Gas Channel	Gas Multi-Hop	Settlement Overhead	Break-Even?
10	—	—	—	—	Ya/Tidak
50	—	—	—	—	Ya/Tidak
100	—	—	—	—	Ya/Tidak
500	—	—	—	—	Ya/Tidak
1000	—	—	—	—	Ya/Tidak

Tabel 4.4c — Settlement: Every 50 TX, Dispute: No Dispute

TX Count	Gas On-Chain	Gas Channel	Gas Multi-Hop	Settlement Overhead	Break-Even?
50	—	—	—	—	Ya/Tidak
100	—	—	—	—	Ya/Tidak
500	—	—	—	—	Ya/Tidak
1000	—	—	—	—	Ya/Tidak

Tabel 4.4d — Settlement: At Close Only, Dispute: 1 Dispute Mid-Channel

TX Count	Gas No-Dispute	Gas w/ Dispute	Dispute Cost	Dispute Overhead (%)
10	—	—	—	—
50	—	—	—	—
100	—	—	—	—
500	—	—	—	—
1000	—	—	—	—

Tabel 4.4e — Settlement: At Close Only, Dispute: Dispute at Close

TX Count	Gas No-Dispute	Gas w/ Dispute	Dispute Cost	Dispute Overhead (%)
10	—	—	—	—
50	—	—	—	—

100	___	___	___	___
500	___	___	___	___
1000	___	___	___	___

Tabel 4.4f — Fixed Costs (diukur sekali)

Operasi	Gas Cost	Catatan
Channel Open	___	Deploy + initial deposit
Channel Close (cooperative)	___	Both parties agree
Channel Close (unilateral)	___	One party timeout
Dispute Initiation	___	Submit challenge
Dispute Resolution	___	Submit counter-evidence
Settlement (per batch)	___	On-chain state update

4.5 Uji Statistik

- Linear regression: total gas = $a \times \text{tx_count} + b$ per mode. Report slope (marginal cost per TX).
- Break-even: solve tx_count dimana gas_channel < gas_on_chain. Report exact point + CI.
- ANCOVA: gas = $f(\text{mode}, \text{tx_count})$ dengan tx_count sebagai covariate.
- Paired comparison: dispute vs no-dispute pada tx_count yang sama.

4.6 Visualisasi

- Line chart: total gas vs TX count (curves: on-chain, channel close-only, channel every-10, channel every-50, multi-hop). Mark break-even points.
- Bar chart: dispute cost as percentage of total, per TX count.
- Stacked bar: gas breakdown (open + off-chain + settlements + close) per settlement strategy.

TOPIK 3D

Evaluasi Merkle Tree Variants (Standard, Sparse, Verkle, Patricia, Poseidon) untuk State Proof Efficiency

1. Deskripsi

Implementasi dan benchmarking 5 varian Merkle tree dari scratch di Python. Evaluasi proof size, generation/verification time, storage, dan batch proof pada berbagai state size.

2. Infrastruktur

- Python + hashlib + py_ecc + poseidon-hash. Colab gratis. timeit + tracemalloc. In-memory data.

3. Langkah

Pekan 1: Standard + Sparse. Pekan 2: Verkle + Patricia + Poseidon. Pekan 3: Benchmark harness. Pekan 4: Full benchmarking + report.

4. Skenario Uji Coba

4.1 Variabel Tetap

- SHA-256 (default hash), key 32 bytes, value 32 bytes, Verkle branching 256, seed tetap, GC disabled, 10 warm-up ops.

4.2 Variabel Manipulasi

Variabel	Level
Tree Variant	Standard Merkle, Sparse Merkle, Verkle, Patricia Trie, Poseidon-Merkle
State Size	1K, 10K, 100K, 1M entries
Operasi	Build tree, Generate proof (1 entry), Verify proof, Update 1 entry, Batch proof (100 entries)

Total: 5 tree × 4 state size × 5 operasi = 100 skenario × 5 rep = 500 run. Per run <5 detik. Total: ~2 jam.

4.3 Variabel Respon

Variabel	Satuan	Pengukuran
Build Time	ms	Waktu bangun tree dari scratch
Proof Generation Time	ms	Generate 1 membership proof
Proof Verification Time	ms	Verify 1 proof
Proof Size	bytes	Ukuran proof dalam bytes
Tree Storage Size	MB	Memory untuk tree (tracemalloc)
Update Time	ms	Update 1 entry + recompute root

Batch Proof Size	bytes	Proof untuk 100 entries sekaligus
Batch Verify Time	ms	Verify batch proof

4.4 Template Pengumpulan Data

Tabel 4.4a — State Size: 1K entries

Metrik	Standard	Sparse	Verkle	Patricia	Poseidon
Build Time (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Gen (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Size (bytes)	___	___	___	___	___
Storage (MB)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Update (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Batch Proof Size (bytes)	___	___	___	___	___
Batch Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___

Tabel 4.4b — State Size: 10K entries

Metrik	Standard	Sparse	Verkle	Patricia	Poseidon
Build Time (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Gen (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Size (bytes)	___	___	___	___	___
Storage (MB)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Update (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Batch Proof Size (bytes)	___	___	___	___	___
Batch Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___

Tabel 4.4c — State Size: 100K entries

Metrik	Standard	Sparse	Verkle	Patricia	Poseidon
Build Time (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Gen (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Size (bytes)	___	___	___	___	___
Storage (MB)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Update (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Batch Proof Size (bytes)	___	___	___	___	___
Batch Verify (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___

Tabel 4.4d — State Size: 1M entries

Metrik	Standard	Sparse	Verkle	Patricia	Poseidon
Build Time (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___
Proof Gen (ms)	___ ± ___	___ ± ___	___ ± ___	___ ± ___	___ ± ___

Proof Verify (ms)	__±__	__±__	__±__	__±__	__±__
Proof Size (bytes)	__	__	__	__	__
Storage (MB)	__±__	__±__	__±__	__±__	__±__
Update (ms)	__±__	__±__	__±__	__±__	__±__
Batch Proof Size (bytes)	__	__	__	__	__
Batch Verify (ms)	__±__	__±__	__±__	__±__	__±__

4.5 Uji Statistik

- Two-way ANOVA: tree_variant × state_size pada setiap metrik.
- Log-linear regression: $\text{proof_size} = a \times \log(\text{state_size}) + b$ per variant — validasi theoretical $O(\log n)$.
- Friedman test: 5 variants pada metrik yang sama.
- Post-hoc Nemenyi: identifikasi pasangan signifikan.

4.6 Visualisasi

- Line chart: proof size vs state size per variant (log-log scale).
- Line chart: build time vs state size per variant (log scale x-axis).
- Grouped bar chart: proof gen/verify time per variant pada 100K entries.
- Bar chart: batch proof size advantage (Verkle vs others) per state size.